

Programación declarativa

- Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

3. Tuplas

Tuplas

```
primero' :: (Int, Int) -> Int
primero' (x, y) = x
```

- (Int, Int) representa el tipo de tuplas con dos elementos de tipo Int.
- (3, 4) es la tupla con los elementos 3 y 4.

Mejor:

```
primero :: (a, b) -> a
primero (x, _) = x
```

- Esta función está predefinida como `fst`.
- Los tipos de datos de los componentes de una tupla pueden ser distintos: `a` y `b`.
- Lo que aparece tras el nombre de la función es un patrón.
- La *variable anónima* `_` se usa cuando no nos interesa ese valor.

Puede haber tuplas de cualquier cantidad (> 1) y tipo de elementos:

```
punto :: (Float, Float, Float)
punto = (0.0, 1.0, 8.5)

persona :: (Int, String, Int, String) -- id, nombre, edad, origen
persona = (12345678, "Pepa", 22, "la Tierra")
```

Comparación de tuplas

- Dos tuplas del mismo tipo pueden compararse si sus componentes pueden compararse.

```
cualEsMenor :: (Int, Char) -> (Int, Char) -> String
cualEsMenor p p' = if p < p' then "el primero"
                  else if p == p' then "iguales"
                  else "el segundo"
```

- `cualEsMenor (1, 'a') (0, 'b') ~> "el segundo"`
- No necesitamos definir `<` y `==` para nuestras tuplas.
 - Haskell los deriva del orden de `Int` y de `Char`, comparando lexicográficamente:
 - * primero la primera componente y, si empatan, la siguiente.
- Tiene sentido hablar de tuplas de funciones:


```
| (fst, not, 3) :: ((a, b) -> a, Bool -> Bool, Int)
```
- Pero las tuplas de funciones no pueden compararse (ni con `==` ni con `<`) porque las funciones tampoco.

Funciones predefinidas sobre tuplas

```
| fst :: (a, b) -> a
| snd :: (a, b) -> b
```

- `fst` y `snd` solo valen para pares (no para tuplas más largas).

Ejercicios

1. Escribe una función que calcule el producto escalar de dos vectores de $\mathbb{R}^3$.
2. Programa una función que calcule cuántos años enteros, días restantes enteros, horas restantes enteras, minutos restantes enteros y segundos restantes hay en una cantidad dada de segundos. Para simplificar, considera que todos los años tienen 365 días.
3. Completa la cabecera y la definición de esta función:

```
| sumaCosas :: ... -> Int
| sumaCosas t = ...

| suma :: Int
| suma = sumaCosas ((1, (2, 5)), (4, 1)) -- suma = 1+2+5+4+1 = 13
```

4. Escribe todas las definiciones posibles para funciones con el tipo (sin restricciones de tipos):
 - `f :: (a, b) -> (b, a)`